A State-Based Imitation Learning Pipeline for Bimanual Box Pickup on the Unitree G1 Humanoid Robot Using ACT-LSTM in MuJoCo Simulation

A Thesis by

Prince Charles Q. Aniñon
Jemarie Fernandez

Submitted to the Department of Computer Science
College of Computing and Information Sciences (CCIS)
Caraga State University – Main Campus

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Science in Computer Science (BSCS)

April 2026

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION

1.1 Background of the Study

Robotics is now one of the fastest growing areas of modern technology, with intelligent robotic systems being developed to carry out tasks that were previously thought to be only possible to be done by humans. As Sun et al. (2025) noted, the recent advances in artificial intelligence and machine learning have changed the course of research in robotics towards data-driven control systems, where robots can acquire complex behaviors through experience, rather than strict rule-based control systems. This has been especially effective in humanoid robotics, which specializes in machines that mimic the structure and movement of the human body to be able to perform and act in human-designed environments, including homes, hospitals, factories, and open areas. Humanoid robotics is currently one of the most vibrant research areas in computer science and engineering due to the increasing need to have robots that can work with humans.

One of the key problems in humanoid robotics is to train robots to do physical manipulation tasks like picking up, moving and placing objects. Conventional programming languages have engineers manually define all the motion and decision rules, which is time-consuming and hard to generalize to other tasks or environments. In order to overcome this shortcoming, scientists have resorted to imitation learning, a paradigm where robots learn by watching and imitating demonstrations of human operators rather than through explicit programming (Zare et al., 2024). Imitation learning has demonstrated significant potential in robotic

manipulation due to its ability to enable robots to learn complex behaviors with relatively few demonstrations. Behavioral cloning is one of its variants that address the learning problem as a form of supervised learning, directly relating observed states to actions based on human demonstrations recorded (Hussein et al., 2017). Nonetheless, it is known that standard behavioral cloning is prone to compounding errors in long sequences of actions, in which small errors in prediction at each time step compound over time and lead to a policy that is no longer representative of the training distribution. This weakness has recently been overcome with architectural improvements like Action Chunking with Transformers (ACT), which anticipates sequences of future actions rather than individual actions and generates more fluent and predictable robot behavior (Zhao et al., 2023).

Despite the progress brought by ACT in addressing compounding errors, a gap remains in how existing imitation learning architectures handle long-horizon, multi-stage manipulation tasks on full humanoid robots. Standard behavioral cloning and ACT-based policies have been applied primarily to short-horizon tabletop manipulation with fixed-base robot arms, and their application to bimanual loco-manipulation on a full humanoid robot in a fully simulation-based pipeline has not been explored. In such tasks, the robot must execute a sequence of distinct phases in a fixed order, including walking toward a target, grasping it with both arms, transporting it, and placing it at a goal location, each requiring different behavioral modes and temporal coordination between locomotion and manipulation. Existing imitation learning systems that address tasks of this scope either require physical robot hardware for data collection and evaluation, or rely on specialized teleoperation equipment that is not feasible in purely simulation-based research workflows.

Long Short-Term Memory networks have been widely recognized for their ability to learn temporal associations in sequential tasks, while Action Chunking with Transformers provides temporally consistent multi-step action prediction for coordinated manipulation (Hochreiter and Schmidhuber, 1997; Zhao et al., 2023). Whether combining these two architectures into a hybrid ACT-LSTM model provides measurable benefit for bimanual loco-manipulation on a full humanoid robot is an open empirical question that has not been previously investigated in a simulation-based pipeline.

To fill this gap, this study proposes a state-based imitation learning pipeline for bimanual box pickup on the Unitree G1 humanoid robot, evaluated entirely within the MuJoCo physics simulation environment. The pipeline introduces a hybrid ACT-LSTM policy architecture in which an LSTM encoder maintains recurrent memory across the full task trajectory while an ACT transformer decoder generates temporally consistent action chunks for coordinated bimanual arm control. The ACT-LSTM policy is compared against a standard behavioral cloning baseline to empirically evaluate whether the hybrid architecture provides measurable benefit on this class of task. Demonstration data is collected through a ZED stereo depth camera that tracks the body movement of a human demonstrator, whose captured motion is translated into robot joint commands through geometric scaling and inverse kinematics within MuJoCo simulation, removing the need for physical robot hardware or specialized teleoperation equipment.

1.2 Statement of the Problem

Standard behavioral cloning methods are prone to compounding errors and covariate shift, causing policies to drift from the training distribution over time, particularly in long-horizon tasks such as bimanual loco-manipulation (Hussein et al., 2017; Zare et al., 2024). While Action Chunking with Transformers has been proposed to address compounding errors through multi-step action prediction, whether

augmenting it with a recurrent LSTM encoder provides additional benefit for bimanual loco-manipulation on a full humanoid robot remains an open empirical question. Beyond this architectural gap, no existing imitation learning pipeline for this class of task combines vision-based teleoperation using accessible depth-sensing hardware with simulation-only training and evaluation, as current systems rely on physical robot hardware or specialized teleoperation equipment that limits their applicability to simulation-based research workflows. This study addresses both gaps by designing and evaluating a state-based imitation learning pipeline for bimanual box pickup on the Unitree G1 humanoid robot in MuJoCo simulation, comparing a hybrid ACT-LSTM policy against a standard behavioral cloning baseline using demonstrations collected via a ZED stereo depth camera teleoperation system.

In particular, the study aims to respond to the following questions:

1. How well does the trained ACT-LSTM policy perform bimanual box pickup and placement based on human demonstrations in MuJoCo simulation, in terms of phase-level task success rate?
2. How does the ACT-LSTM policy compare against a standard behavioral cloning baseline in terms of phase-level task success rate on box positions encountered during demonstration collection?
3. How well do the ACT-LSTM policy and the behavioral cloning baseline generalize to box positions not encountered during demonstration collection, in terms of phase-level task success rate?

1.3 Objectives of the Study

The overall goal of the proposed research is to design and evaluate a state-based imitation learning pipeline that enables the Unitree G1 humanoid robot to learn and perform bimanual box pickup based on human demonstrations in the MuJoCo simulator, comparing a hybrid ACT-LSTM policy architecture against a standard behavioral cloning baseline.

In particular, the study will have the following objectives:

1. To design and implement a teleoperation-based demonstration collection system using a ZED stereo depth camera that captures human bimanual motion and translates it into robot joint commands within MuJoCo simulation.
2. To develop and train a hybrid ACT-LSTM imitation learning policy and a standard behavioral cloning baseline for bimanual box pickup and placement using the collected demonstration dataset.
3. To evaluate and compare the performance of the ACT-LSTM policy and the behavioral cloning baseline on box positions encountered during demonstration collection in terms of phase-level task success rate.
4. To evaluate and compare the spatial generalization of the ACT-LSTM policy and the behavioral cloning baseline on box positions not encountered during demonstration collection.

1.4 Significance of the Study

This study contributes to the growing body of research on simulation-based imitation learning for humanoid manipulation, with a specific focus on the application of a hybrid ACT-LSTM policy architecture to bimanual loco-manipulation on a full humanoid robot. The findings and outputs of the study are expected to benefit the following groups.

1. **Robotics and artificial intelligence researchers** will benefit from the empirical evidence on how a hybrid ACT-LSTM architecture performs on a bimanual loco-manipulation task in a fully simulation-based pipeline. The phase-level evaluation results provide measurable insight into how effectively the proposed architecture learns and executes sequential multi-phase manipulation behavior from human demonstrations, contributing to ongoing research on policy learning for humanoid manipulation.

2. **Robotics students and academic researchers** will benefit from the proposed pipeline as a practical and reproducible framework for studying imitation learning for humanoid manipulation. The fully simulation-based design makes it possible to conduct meaningful humanoid manipulation research using simulation software and depth-sensing hardware without requiring physical robot deployment.
3. **Academic institutions and robotics laboratories** can use the pipeline architecture, demonstration collection methodology, and policy learning framework as a starting point for imitation learning research conducted within a simulation-based environment. The structured design of the pipeline makes it adaptable for institutions that are building research capacity in humanoid robotics.
4. **Future researchers** can use the evaluation results of this study as a reference for subsequent work on more complex manipulation tasks, alternative policy architectures, or the transfer of learned policies to physical robot execution. The pipeline also serves as a reproducible baseline for future studies that investigate hybrid temporal architectures in simulation-based loco-manipulation research.

1.5 Scope and Limitation of the Study

This study aims to build a state-based imitation learning pipeline for bimanual box pickup on the Unitree G1 humanoid robot. The pipeline consists of four main components: human demonstration collection via teleoperation with a ZED stereo depth camera, state and action representation via privileged simulator observations, temporal policy learning via a combined Action Chunking with Transformers and Long Short-Term Memory (ACT-LSTM) architecture, and performance evaluation in the MuJoCo physics simulation environment. Data on human demonstration will be collected by live teleoperation where the movements of the arms of the demonstrator observed by the ZED camera are converted into end-effector targets

that drive the robot arms, and the movements of the pelvis of the demonstrator observed by the same camera are converted into velocity commands sent to a pre-trained walking policy from the unitree_rl_gym repository. Bimanual grasping will be triggered by tracking target positions defined relative to the box center in the simulation. Around 100 to 150 demonstration episodes will be gathered with variability in box position to support generalization testing. Phase-level task success rate will be used to determine performance in a quantitative manner.

In this work, real-time control and deployment on a physical robot will not be included. All evaluation will be done in simulation only, and the policy will rely on simulator state observations rather than camera input during autonomous deployment. The task is limited to a fixed size box in a fixed orientation, picked up at different positions within the robot's workspace and placed at a goal location. Generalization testing will only cover changes in box position and may not apply to different box sizes, orientations, or task configurations outside the training distribution. The dexterous fingers of the Unitree G1 will not be used, as grasping is handled through simplified gripper-level control rather than individual finger movements. Other learning approaches such as reinforcement learning and locomotion learning from demonstrations are outside the scope of this work. Environmental conditions such as lighting and visual obstruction that may affect ZED body tracking during demonstration collection will not be independently tested or addressed in this study.

CHAPTER 2. REVIEW OF RELATED LITERATURE

2.1 Theoretical Framework

This study is situated at the intersection of computer vision, machine learning, robotics, and kinematics. The fundamental theoretical concept underpinning this study is that of state-based imitation learning, which deals with training robots in replicating certain behaviors exhibited by human demonstrations without any need for manually programming or designing appropriate rewards (Zare et al., 2024; Sun et al., 2025). The architecture of this pipeline rests on four theoretical foundations, each independently underlining one part of the framework.

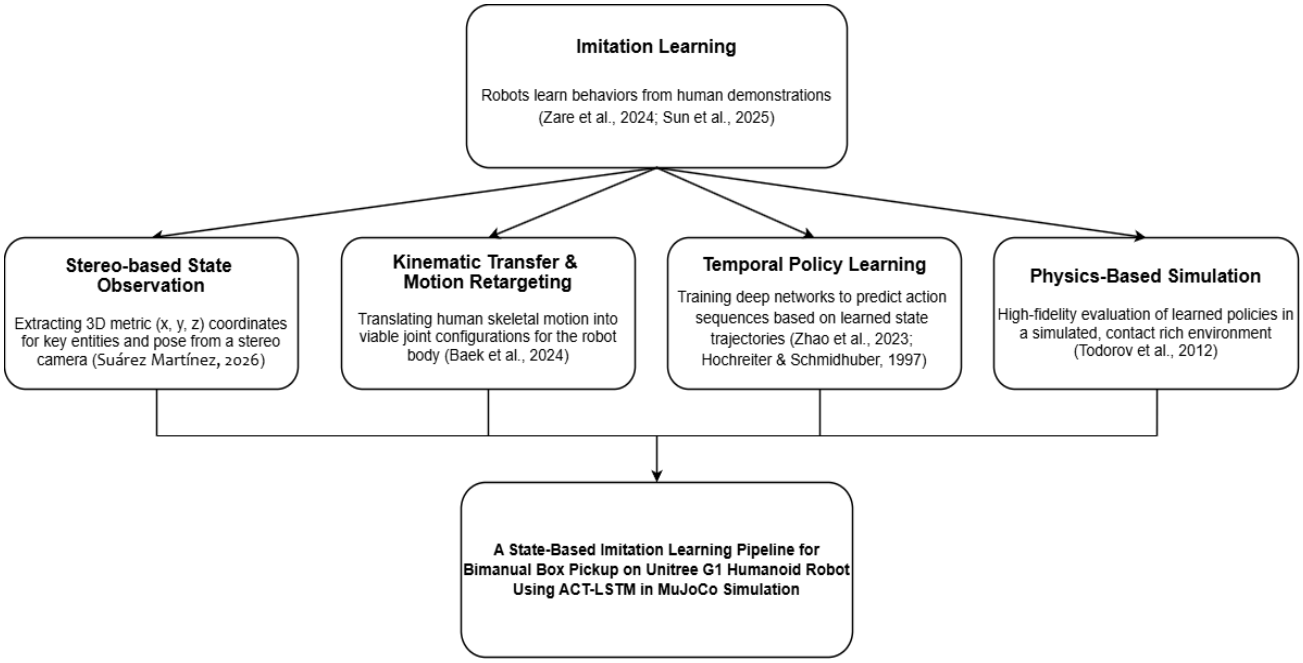

Figure 2.1. *Theoretical Framework of the Proposed State-Based Imitation Learning Pipeline*

Figure 2.1 shows the theoretical framework of the proposed study. Inspired by the vision-based depth sensing concept adopted by Suárez Martínez (2026), pose estimation using stereo cameras offers the necessary starting point for recovering accurate 3D body keypoints based on the human demonstrations performed in

metric space. While Suárez Martínez utilized the Luxonis OAK-D Pro, this study employs the ZED stereo camera as the depth sensing hardware. Kinematic retargeting refers to the problem defined by Baek et al. (2024) regarding the transfer of the recovered spatial data to a completely different body and different set of joints via the use of Inverse Kinematics (IK). The temporal model of learning encompasses two complementary approaches: the Action Chunking with Transformers (ACT) model described by Zhao et al. (2023), which generates consistent multi-step action sequences, and Long Short-Term Memory (LSTM) networks (Hochreiter & Schmidhuber, 1997), which maintain recurrent phase-level memory across the full task trajectory. Finally, simulation is based on the MuJoCo physics engine introduced by Todorov et al. (2012) that serves as an environment for evaluating the entire pipeline. These four pillars define the theoretical background behind the state-based imitation learning pipeline for the Unitree G1 humanoid robot.

2.2 Imitation Learning and Behavioral Cloning

Robotics is experiencing a major paradigm shift in classical rule-based and model-based control to the learning-based approach. Conventional control systems represent the robot and its environment explicitly in the form of mathematical equations that are meant to produce control signals. Even though these approaches have been rather effective in a limited industrial setting, they are naturally fragile and not as adaptable as required to address the challenges of the modern humanoid systems (Kuindersma et al., 2016). The peculiarities of a humanoid system are high degrees of freedom, complex interactions between contacts, and dynamic stability, which make it impossible to develop a precise mathematical model (Sun et al., 2025). Model Predictive Control and Whole-Body Control may be considered the most advanced classical methods of controlling humanoid motions, but they are very fragile in unstructured conditions and require a lot of manual fine-tuning of cost functions (Kuindersma et al., 2016). The learning-based methodologies circumvent the aforementioned drawbacks by learning control policies from data. Imitation

learning can be considered the most practical approach for training a humanoid to perform complicated bimanual tasks.

Imitation learning (IL) is a family of methods that enable robots to acquire skills by observing and replicating expert demonstrations, bypassing the need for manually specified trajectories or reward functions (Zare et al., 2024). The basic premise of imitation learning is that the task structure is implicitly encoded in the sequences of states and actions observed in the demonstration, and that there is a policy that can be trained on such data to discover such a structure. IL is therefore very attractive when designing robots to accomplish complex tasks for which designing a meaningful reward signal is impossible; e.g., for precise and complicated manipulation involving multiple limbs.

Behavioral Cloning (BC) is the most foundational IL method. It casts the problem of imitation learning as a regression problem where, given a set of state-action pairs from an expert (s,a), a policy π is trained to predict the actions that minimize the prediction error with respect to those of the expert (Hussein et al., 2017). Despite the ease with which the problem can be posed, BC is known to fail spectacularly owing to the problem of accumulating errors and covariate shifts.

Covariate shift arises due to the fact that the policy trained by BC on states that arise while the agent performs the tasks based on expert demonstrations is executed in a completely new environment, namely, the states resulting from the policy itself (Ross & Bagnell, 2010). The agent drifts off from the demonstration states, which results in entering new states that the policy has never been seen before and therefore has no knowledge about recovering from such situations, resulting in increasingly poor decisions made by the policy. These mistakes will continue accumulating until the agent finally reaches a state where it cannot recover anymore. This is especially true for the long-term and bimanual tasks like box picking, which require perfectly symmetric arms. Zare et al. (2024) empirically proved that regular BC agents quickly degrade their performance on tasks with a larger horizon.

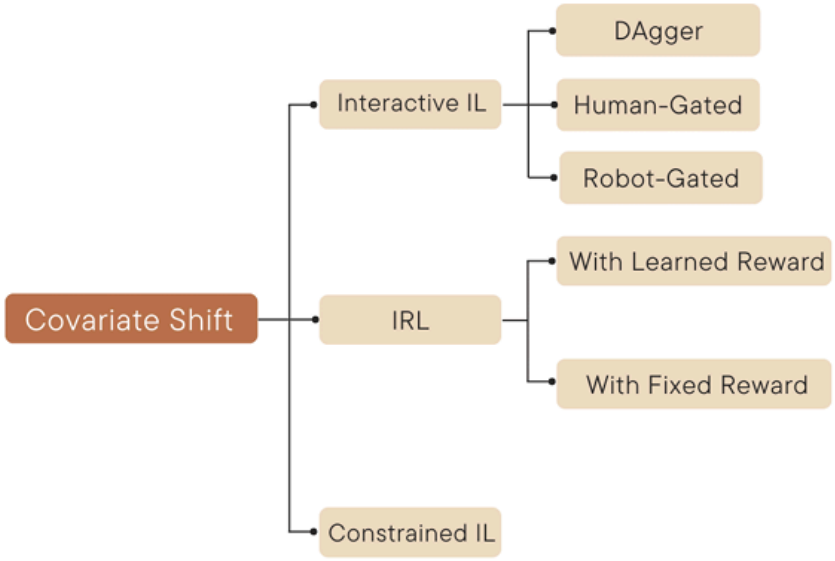

***Figure* 2.2.** *Illustration of covariate shift and compounding error in Behavioral Cloning. Retrieved from Ross & Bagnell (2010).*

2.3 Temporal Modeling in Imitation Learning

A foundational challenge motivating temporal modeling in imitation learning is that human demonstration data is inherently non-Markovian. Mandlekar et al. (2021) established empirically that human demonstrators' actions are influenced not only by the current observation but by the history of prior states and actions, meaning that any policy conditioned solely on the current observation is structurally mismatched to the demonstrations it trains on. As Zhou et al. (2025) formalize, such policies face a Partially Observable Markov Decision Process (POMDP) setting where identical observations can arise from fundamentally different task histories, yet the policy has no mechanism to distinguish between them. This misalignment is not merely a generalization problem but an architectural one, and it motivates the need for temporal memory as a first-class design requirement in imitation learning for sequential manipulation tasks.

Standard behavioral cloning models predict each step separately, failing to account for the temporal interdependence that arises due to physical actions. Motion

in humans is inherently continuous and phase-related; reaching, picking up, and lifting are not separate actions but steps of one cohesive action. Any policy model that does not take this structure into account will make mistakes at cross-frames, leading to jitter and unreliable joint velocities, which is highly dangerous to a humanoid robot, particularly with regard to its balancing capabilities (Graßhof et al., 2023). Long Short-Term Memory (LSTM) networks of Hochreiter and Schmidhuber (1997) were developed precisely to address this by maintaining a gated memory cell that is updated at each timestep, allowing the network to carry relevant information forward across arbitrarily long sequences rather than treating each input as an isolated observation. The gating mechanism — comprising input, forget, and output gates — allows the network to learn which information from prior timesteps is relevant to the current decision and which can be discarded, making LSTM structurally capable of accumulating evidence about which phase of a task has been completed even when individual observations within each phase appear perceptually similar (Hochreiter & Schmidhuber, 1997). Beyond robotic manipulation, the effectiveness of LSTM's recurrent architecture has been demonstrated across a wide range of sequential prediction domains, including speech recognition, where deep recurrent networks significantly outperformed frame-independent models at the time of their introduction (Graves et al., 2013). Graßhof et al. (2023) demonstrated that LSTM-based architectures substantially outperform frame-independent models at forecasting human movement across long-horizon sequences, confirming the practical value of recurrent memory when historical context is needed for accurate sequential prediction.

This capacity for phase-level tracking has been empirically confirmed in robotic manipulation settings. Mandlekar et al. (2021) demonstrated that BC-RNN — a behavioral cloning policy with a recurrent hidden state — substantially outperformed non-recurrent baselines on multi-stage manipulation tasks, with the performance gap growing to approximately 55% on longer-horizon tasks where accumulated phase context becomes most critical, leading the study to conclude that

algorithms incorporating some notion of temporality are consistently more effective when learning from human demonstration data. Pirk et al. (2021) further confirmed that an LSTM-based encoder can correctly infer which phase of a long-horizon sequential manipulation task is currently active from accumulated observation history alone, even when individual observations within each phase are perceptually similar. Rahmatizadeh et al. (2018) demonstrated that stacked LSTM networks trained on simulation demonstrations implicitly track task progress across timesteps in manipulation pipelines, confirming that recurrent memory encodes sequential task structure from demonstration data without requiring explicit phase labeling.

However, phase-level memory alone does not constitute a sufficient temporal model for complex bimanual execution, because LSTM is structurally incapable of guaranteeing consistency across the next several timesteps. LSTM networks are fundamentally autoregressive: they emit one action per timestep, and each prediction is conditioned on the previous hidden state rather than on a committed future plan. This has two consequences that are particularly damaging for bimanual tasks. First, LSTM provides no mechanism for enforcing coherence across consecutive predictions, meaning that coordinated two-arm motions — which require both limbs to commit to a shared trajectory segment simultaneously — can diverge frame-by-frame. Grannen et al. (2023) establish that reliable bimanual manipulation requires each arm to simultaneously commit to a shared trajectory segment, a property that frame-level autoregressive prediction cannot satisfy since each step is generated independently without commitment to a future plan. Second, because errors at time t corrupt the hidden state passed to time t+1, prediction mistakes compound forward through the trajectory rather than being absorbed. On a task such as symmetric box-grasping, where both arms must converge on a shared object simultaneously, this frame-level instability is structurally incompatible with reliable execution, and no amount of recurrent capacity resolves it because the problem is architectural rather than parametric.

Action Chunking with Transformers (ACT), developed by Zhao et al. (2023), presents a radically different perspective on temporal modeling that explicitly addresses the shortcomings of single-step autoregressive prediction described above. Instead of generating one action per timestep, ACT uses a transformer-based encoder-decoder model to generate an entire sequence — a "chunk" — of future actions based on the present observation. This commitment to a consistent action sequence as opposed to isolated actions means that ACT guarantees temporal consistency throughout the entire prediction window, making erratic behavior extremely rare. According to Zhao et al. (2023), ACT outperformed traditional BC and other imitation learning baselines on bimanual manipulation tasks by up to 59%, with performance gains attributed to action chunking reducing the effective task horizon and temporal ensembling smoothing policy execution. Temporal ensembling — described in operational detail in Section 3.6 — is an inference-time technique in which the robot executes a running weighted average over all currently active action chunks rather than committing entirely to any single chunk's prediction; because multiple overlapping chunks generated at slightly different timesteps are blended together, discontinuities at chunk boundaries are dampened, producing joint trajectories that are smoother and more robust to individual prediction errors (Zhao et al., 2023). The CVAE objective used to train the ACT policy, first introduced by Sohn et al. (2015), allows the model to capture the multimodal variability present across human demonstrations by encoding demonstration-level style into a latent variable during training, which is marginalized out at deployment for deterministic prediction.

Yet ACT carries its own structural limitation that is symmetrical to LSTM's weakness. Its transformer encoder operates over a fixed, short observation window and has no recurrent mechanism that persists information across that window's boundary, meaning ACT has no intrinsic representation of where in the task the robot currently is. Given an identical short observation window, ACT cannot distinguish whether the robot is in the approach phase, the grasp phase, or the placement phase, because all such windows may contain visually similar joint configurations.

Without phase context, the action chunks it produces — though internally consistent — may be misaligned with the actual stage of the trajectory, rendering their internal consistency insufficient for reliable multi-phase execution. Zhou et al. (2025) confirmed this experimentally on a four-stage sequential manipulation task: ACT, relying only on immediate context, was unable to distinguish the observation state after completing one stage from the state after completing a later stage, causing it to skip entire required phases and achieve zero success on the overall task — confirming that temporal ambiguity is a categorical architectural failure rather than a matter of insufficient training or data. Mandlekar et al. (2021) provide corroborating evidence at benchmark scale, showing that non-recurrent history-free architectures are consistently outperformed by history-dependent ones on multi-stage tasks, with the performance gap growing larger as task horizon increases. Critically, this limitation cannot be resolved by simply extending ACT's observation window. Mark et al. (2026) demonstrate that enlarging the context window of short-history policies incurs prohibitive computational and memory costs and causes overfitting to spurious temporal correlations, leading to catastrophic failures under distribution shift — confirming that the problem is architectural rather than one of insufficient context length, and that no parameter-level adjustment can substitute for a dedicated recurrent mechanism.

Because these deficiencies are complementary rather than overlapping, neither model alone can satisfy the requirements of a multi-phase bimanual task. LSTM can identify the current phase but cannot commit to a coordinated multi-step plan; ACT can commit to a coordinated multi-step plan but cannot identify which phase warrants it. A mechanism that addresses both simultaneously is therefore architecturally necessary.

This complementarity motivates the proposed hybrid ACT-LSTM architecture, in which the LSTM serves as the phase encoder and ACT serves as the chunk decoder. At each decision point, the LSTM encoder processes the full trajectory history and produces a phase-aware hidden state capturing which stage of the task is currently

being executed; this hidden state is then passed to the ACT transformer decoder as contextual input, conditioning chunk generation on the current phase rather than on the short observation window alone. The result is an architecture in which phase disambiguation and action consistency are handled by separate, purpose-built components: LSTM resolves when in the task the robot is, and ACT resolves what sequence of actions is appropriate for that moment. Wu et al. (2025) independently confirmed that recurrent memory is the established solution category for state ambiguity in multi-stage sequential manipulation, validating the role assigned to the LSTM encoder in this architecture. The concept of combining recurrent encoders with transformer decoders has clear precedent in multiple fields. In natural language processing, architectures pairing LSTM encoders with transformer-based attention decoders improved sequence transduction tasks over either component alone (Chen et al., 2018). In multivariate time series forecasting, hybrid recurrent-transformer models have consistently demonstrated superior performance over purely recurrent or purely attention-based baselines, capturing both local temporal dynamics and long-range dependencies simultaneously (Wu et al., 2021; Zhou et al., 2021). In robotic policy learning, attempts to pair recurrent history encoders with transformer-based decoders to extend temporal context have similarly been explored (Jang et al., 2022). However, to the best of the authors' knowledge, no such architecture has been applied to bimanual loco-manipulation on a humanoid platform, making the ACT-LSTM model a novel contribution of the proposed study.

This division of responsibility reflects an architectural recognition that temporal modeling in a multi-phase bimanual task involves two distinct and non-substitutable problems — phase tracking across a long horizon and coherent multi-step planning within a phase — that are best addressed by components with correspondingly distinct inductive biases. For the specific task of box picking with a bimanual humanoid robot, which requires the robot to perceive and execute four separate phases — walking approach, bimanual pickup, box transport, and box placement — neither model is sufficient on its own: without recurrent memory, ACT

cannot distinguish the current phase of execution from a short observation alone, and without ACT's action chunking, LSTM cannot provide the temporal consistency required for coordinated bimanual arm trajectories. The recurrent structure of LSTM is suited to cumulative state integration over time, while the attention mechanism of the transformer is suited to structured, globally consistent sequence generation, and it is precisely this pairing of complementary inductive biases that makes the hybrid ACT-LSTM model a principled rather than incidental architectural choice and a novel contribution of the proposed study.

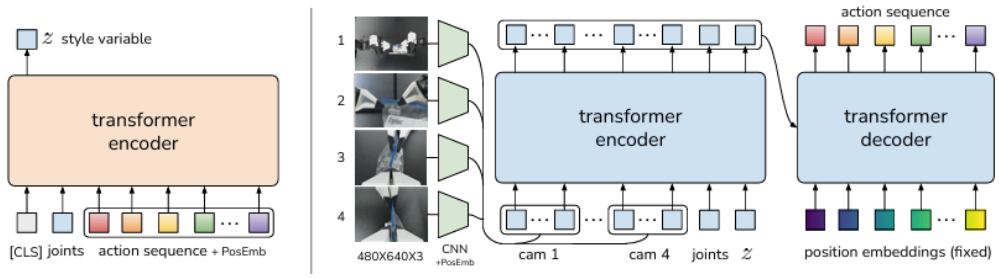

Figure 2.3. *ACT Transformer encoder-decoder architecture for action chunking in bimanual manipulation. Retrieved from Zhao et al. (2023).*

In the proposed study, these two methods are integrated into a single model termed the hybrid ACT-LSTM model, wherein the transformer performs multi-step action prediction while a stacked LSTM encoder maintains phase-wise memory throughout the task trajectory. This fusion is made particularly for the purpose of solving this specific task because of its sequential multi-phase structure and bimanual coordination requirements.

2.4 Demonstration Collection: Teleoperation and Motion Retargeting

The availability and quality of human demonstration data are two key factors for policy efficacy in imitation learning setups. In order to use the demonstration data for training a manipulation policy, two challenges need to be addressed: one involves capturing the movements of a human accurately, while the other involves transferring those movements to the robot body with a different geometry. The following is an outline of the methods used in addressing both challenges.

Demonstration Collection Methods

Three primary approaches are used in the literature to acquire human demonstration data for robot manipulation: motion capture, teleoperation, and vision-based depth sensing.

Motion capture technologies embody the proven benchmark of spatial precision in terms of acquiring the human posture. Systems like Vicon and OptiTrack employ infrared cameras to triangulate the location of reflective markers placed on the demonstrator's body. Nevertheless, the monetary and technical investment required from the use of motion capture technology, including multi-camera infrastructure, calibration demands, and system licensing fees, places it financially out of reach for many research institutions, particularly in developing countries (Singh et al., 2024).

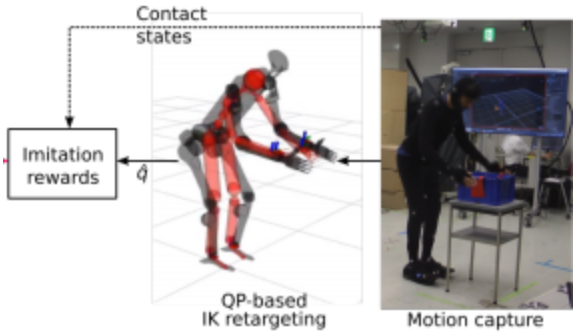

Figure 2.4. *Example of a commercial motion capture setup for robotic demonstration collection. Retrieved from Singh et al. (2024).*

Teleoperation interfaces, which consist of a human controlling the robot by shown to collect excellent bimanual demonstrations using teleoperated methods (Zhao et al., 2023; Chi et al., 2024) . But teleoperated methods do necessitate access to the robot physically when collecting data, which may not be possible in a purely simulated pipeline. Operator artifacts, including hesitation and correction behaviors, have been shown to corrupt the data collected via teleoperation (Chi et al., 2024).

Vision-based depth sensing presents an affordable option. While monocular pose estimation methods that derive depth estimates from a single RGB frame using neural networks have proven effective for skeletal tracking, their performance is constrained by depth ambiguity, generating relative instead of metric depth values (Seo et al., 2023). This creates a challenge in retrieving exact 3D hand locations in metric space, which is necessary in bimanual manipulation tasks since the exact spatial positioning of both hands and the target object is imperative. Stereo depth cameras address this issue by calculating disparity based on two spatially distinct image sensors, resulting in metrically calibrated 3D locations. Seo et al. (2023) showed that stereo skeleton tracking algorithms can achieve near entry-level markerless motion capture accuracy without any specialized hardware requirements for the demonstrator. The StereoLabs ZED 2i camera adopted in the proposed pipeline performs real-time 3D body tracking through integration of neural pose detection with stereoscopic depth processing, enhancing joint localization in

occluded scenarios (StereoLabs, 2024) . In view of the accessibility aims and simulated nature of the research, the ZED 2i sensor is the optimal choice for data gathering.

Figure 2.5. StereoLabs ZED 2i stereo depth camera used for real-time 3D body tracking in the proposed pipeline. Retrieved from StereoLabs (2024)

Motion Retargeting via Inverse Kinematics

Capturing human pose is only the first step. Because the human demonstrator and the Unitree G1 humanoid robot have different body proportions and joint configurations, the captured motion cannot be applied directly to the robot. Motion retargeting is the process of transferring human motion onto a kinematically different robot body while preserving the spatial intent of the original movement (Baek et al., 2024).

The central challenge in retargeting is known as the correspondence problem, where a human limb segment of a certain length needs to be matched with a robotic limb segment of another length without deforming the trajectory of the robot's end effector concerning the object involved in the task. The technique of geometric scaling provides an elegant solution for this issue through maintaining the directionality of the human limb segment and adjusting its length to the appropriate robotic limb length (Suárez Martínez, 2026).

Once the scaled target positions have been set, Inverse Kinematics (IK) can be performed to determine the joint angles that would allow for achieving the goal positions of the robot's end-effectors. IK solves the problem of the end-effectors' positioning by repeatedly changing the values of joint angles in such a way that the difference between the calculated current position of the end-effectors based on forward kinematics and their desired positions becomes minimal (Siciliano et al., 2009). Since MuJoCo provides a built-in numerical inverse kinematics solver, it is well suited for computing joint angles for the Unitree G1 within the simulation environment. The approach was confirmed empirically by Suárez Martínez (2026) to provide stable results when reproducing human arm movements on the Unitree G1 platform.

It should be mentioned that although the geometric scaling and IK-based retargeting technique employed in this paper has been previously tested by Suárez Martínez (2026) based on a Luxonis OAK-D Pro RGB-D sensor instead of the ZED 2i, the actual retargeting process itself is independent of the exact type of depth-sensing technology since all metrically calibrated 3D landmark positions in the robot's reference frame are used as input for the geometric scaling and IK pipeline. The Luxonis OAK-D Pro and the StereoLabs ZED 2i provide metric 3D point positions via stereo depth mapping, and hence, the validated retargeting process is directly applicable to the suggested pipeline where the ZED 2i replaces the OAK-D Pro RGB-D camera as the sensory input frontend for landmark detection. An illustration of the results of geometric scaling applied to the right arm of a Unitree G1 robotic platform has been provided by Suárez Martínez (2026).

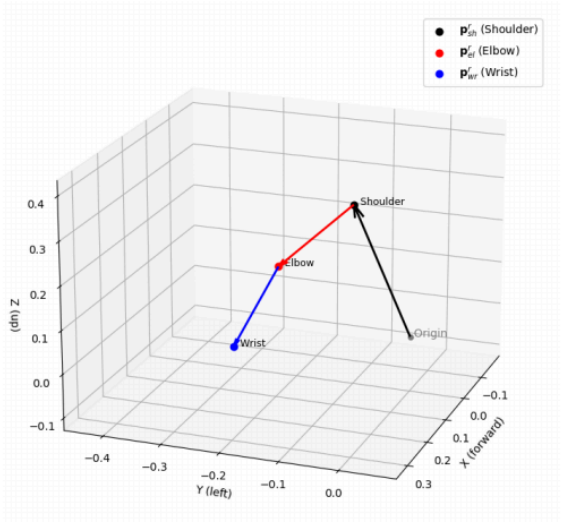

Figure 2.6. *Vector representation of the Unitree G1 right arm after geometric scaling applied to human landmark positions. Retrieved from Suárez Martínez (2026).*

The combination of the ZED-based stereo pose estimation and the IK-based motion transfer algorithm constitutes the system for creating the demonstration database in the proposed method. Keypoints captured by the ZED camera are converted from the camera coordinate system into the robot coordinate system, resized to correspond to the size of Unitree G1 joints, and supplied to the IK algorithm as input to generate joint commands in MuJoCo simulation. A description of this method is provided in detail in section 3.3.

2.5 Simulation Environment

Simulation Physics-based simulation is now a common methodology in the study of humanoid robot learning, offering a safe, scalable and inexpensive platform to develop and test control policies prior to physical implementation. Implementing untested control policies directly on physical equipment creates high risks, such as mechanical breakdown, environmental harm, and high financial expenses. According to Muratore et al. (2022), the latest trends in robot learning are moving towards the simulation setting, where data generation is cheap and rapid, and simulation is becoming a matter of standard and scholarly accepted practices as an alternative to

experimentation with physical hardware. Simulation environments provide a safe and scalable environment in which synthesized kinematic commands can be checked against virtual physical laws, and are especially well-suited to academic research environments with restricted access to hardware (Zhao et al., 2020).

MuJoCo (Multi-Joint dynamics with Contact) is the most popular physics simulation platform, and has become the default environment in humanoid robot studies and imitation learning. MuJoCo, initially created by Todorov et al. (2012) and open-sourced by DeepMind in 2022 (DeepMind, 2022) uses generalized coordinates and a continuous-time optimization problem to simulate complex multi-contact dynamics with high physical fidelity, e.g. the feet of a humanoid robot touching the surface or a robot hand holding an object. It is especially well-suited to the task of computing both forward and inverse dynamics, which is why it is well-suited to the task of assessing humanoid robot motions, where accurate simulation of joint-level dynamics is necessary to determine the feasibility and stability of retargeted motions. Zakka et al. (2022) also extended MuJoCo Menagerie, a set of high-quality simulation models of MuJoCo with detailed humanoid robot models, and which provides the simulation model of Unitree G1 used in the proposed study.

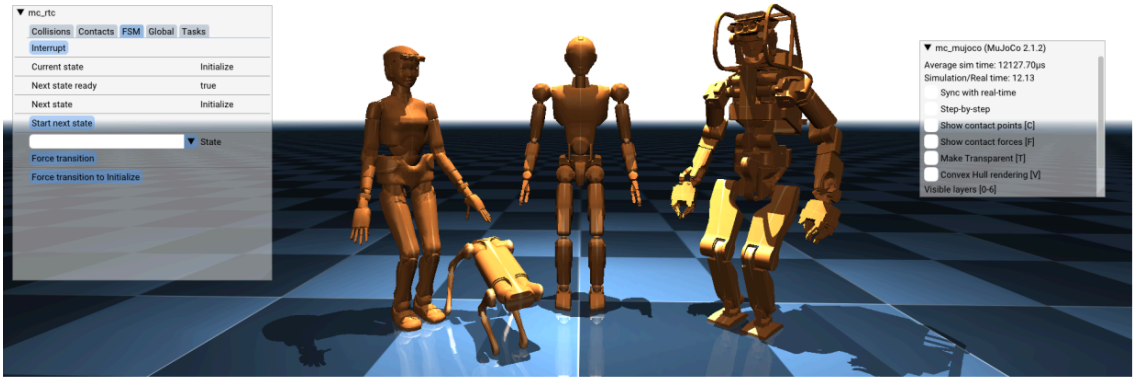

Figure 2.7. *MuJoCo simulator with humanoid robot. Obtained in Singh et al. (2023).*

The Unitree G1 is a humanoid robot with 29 degrees of freedom (depending on configuration) with a height of about 127 centimeters and a weight of 35 kilograms (Unitree Robotics, n.d.). The unitree_rl_gym repository contains an official MuJoCo model of the Unitree G1, making it a convenient and well-documented option in imitation learning studies based on simulation.

Figure 2.8. *Unitree G1 humanoid robot. Retrieved from Unitree Robotics (n.d.).*

Suárez Martínez (2026) showed that Unitree G1 simulation with MuJoCo can be used to test vision-based imitation learning pipelines and show that it can achieve accurate and consistent replication of human arm movements in simulation and on physical hardware. This further validates the use of the Unitree G1 MuJoCo model as the evaluation platform in the proposed study. MuJoCo as an output environment of this study is the environment in which the generated joint trajectories are implemented and tested on the Unitree G1 model. Instead of focusing on actual implementation, the proposed pipeline applies MuJoCo solely in the context of simulation-based testing, in which the retargeted motions are tested to be executable, stable, and similar to the original human demonstrations. This simulation-based method aligns with the current practice in the study of humanoid

robot learning, providing a physically realistic evaluation platform without requiring physical robot deployment.

2.6 Existing IL Pipelines for Bimanual Manipulation

Bimanual manipulation is characterized by its own specific problems in the context of imitation learning, aside from issues that are prevalent when working on one-hand tasks (Zhao et al., 2023; Fu et al., 2024). To work on synchronizing the movement of the two arms, it is necessary to train them in terms of timing and synchronization in space, performing symmetrical or coordinated motion, and handling dual-handed interactions. Numerous studies have been done regarding the development of pipelines for bimanual manipulation, and the most pertinent studies are discussed herein considering three factors: hardware setup for demonstrations, methodology for policy evaluation, and availability of the pipeline within limited resource academic environments.

Notable examples include ALOHA (Zhao et al., 2023), UMI (Chi et al., 2024), and the full-body locomotion manipulation system developed by Sun et al. (2025). The ALOHA method uses the dual-arm teleoperation framework together with the ACT approach, but it needs two physical robot arms of the ViperX type, which cost roughly $20,000 each, with data collection and policy evaluation being directly tied to the use of the physical robots. In turn, the UMI framework lowers certain collection hurdles by using a hand-held gripping device but still relies on physical robots during evaluation, focusing on easier bimanual tasks. Sun et al. (2025) are closest to the current study regarding task scope by learning walking and bimanual arm movements simultaneously through datasets via transformer policies; however, their approach requires extensive demonstrations on physical humanoid hardware with no possibility of simulating it, thus excluding resource-limited organizations from its implementation process. Singh et al. (2024) describe an imitation pipeline for learning locomotion and manipulation tasks based on human guidance using motion capture, obtaining stable behavior on physical hardware in a realistic environment.

Nevertheless, unlike the current study, this system requires costly motion capture setup, physical implementation of a humanoid robot and further assistance from experts.

The simulation-based imitation learning pipeline provides an incomplete solution in that it eliminates the requirement of deploying robots physically for the evaluation of policies. As reported by Zhao et al. (2020), policies learned in simulation alone are capable of producing behaviors that are transferable to real robots in controlled environments, making simulations a proven ground for developing policies through imitation learning. However, as observed from the studies reviewed, current simulation-based imitation learning pipelines cater only to tasks involving a single arm manipulation, require specialized teleoperation equipment, or do not account for all steps in the pipeline.

The consistent and notable gap therefore exists across all previous works reviewed. None of the existing systems combine a method to obtain demonstrations through vision-based teleoperation using cheap sensor hardware without the need for physical robot hardware with a simulation-only method for training and evaluation of policies for bimanual manipulation tasks. Existing systems able to perform strongly on the bimanual manipulation task all make use of some form of robot hardware along the data collection or evaluation process pipelines, while simulation only systems have not yet addressed the issue of collecting demonstrations by employing low-cost sensor hardware. The proposed research project directly addresses the above stated gap through a combination of both elements, which is to be performed within the confines of the MuJoCo simulated environment running on the Unitree G1 robot.

2.7 Synthesis and Research Gap

The literature covered in Sections 2.2 through 2.6 reveals that while significant progress has been made in imitation learning for humanoid manipulation, all of the systems surveyed share at least one requirement that limits their reproducibility in

resource-constrained research environments: the use of physical robot hardware, specialized motion capture systems, or dedicated teleoperation equipment for demonstration collection. This restriction is inherent across all reviewed systems. The process of collecting demonstrations, training policies, and evaluating performance in existing work is built on assumptions that necessitate access to physical robot hardware or specialized sensing infrastructure. Up until now, no previous work has addressed all three stages of this pipeline — demonstration collection, policy learning, and policy evaluation — within a fully simulation-based workflow using accessible sensing hardware combined with a hybrid temporal policy architecture.

A second and independent gap exists at the architectural level. While ACT-based systems have demonstrated strong performance on short-horizon bimanual manipulation tasks, and LSTM-based systems have demonstrated recurrent phase tracking in sequential prediction settings, no existing work has applied a hybrid ACT-LSTM architecture to bimanual loco-manipulation on a full humanoid robot. This represents an uninvestigated combination of architectural components for a class of task that requires both multi-step action consistency for coordinated bimanual arm control and recurrent memory for tracking task progress across sequential phases. The absence of this investigation means that no validated architectural blueprint currently exists for this specific class of problem.

The proposed study addresses both gaps by integrating the four areas reviewed in this chapter into a single coherent and fully reproducible pipeline. The ZED stereo pose estimation technique substitutes motion capture and physical teleoperation for demonstration data acquisition, providing metrically accurate 3D keypoints using accessible off-the-shelf hardware. Through geometric scaling and inverse kinematic mapping, the captured human motion is transferred to the Unitree G1 joint configuration within MuJoCo simulation, resolving the morphological discrepancy between the human demonstrator and the robot. A hybrid ACT-LSTM policy architecture, which has not previously been explored for bimanual humanoid

loco-manipulation, is then trained on the collected demonstrations and evaluated entirely within the same simulation environment. By means of this integration, the proposed pipeline demonstrates that a complete and reproducible imitation learning workflow for bimanual loco-manipulation on a humanoid robot can be designed and evaluated using accessible simulation tools and sensing hardware, without physical robot deployment. Also, through empirical comparison of the hybrid ACT-LSTM architecture against a standard behavioral cloning baseline, this study provides the first quantitative evidence of whether the architectural complexity of ACT-LSTM is justified for this class of task on a full humanoid robot platform.

CHAPTER 3 METHODOLOGY

3.1 Research Design

This study follows an experimental research design in which a state-based imitation learning pipeline is developed and evaluated for bimanual box pickup on the Unitree G1 humanoid robot in MuJoCo simulation. The pipeline is constructed in three sequential phases: demonstration collection through Zed-based teleoperation, ACT-LSTM policy training through behavioral cloning, and autonomous evaluation in MuJoCo simulation. Performance is measured quantitatively using phase-level task success rate as defined in Section 3.8.

3.2 Conceptual Framework

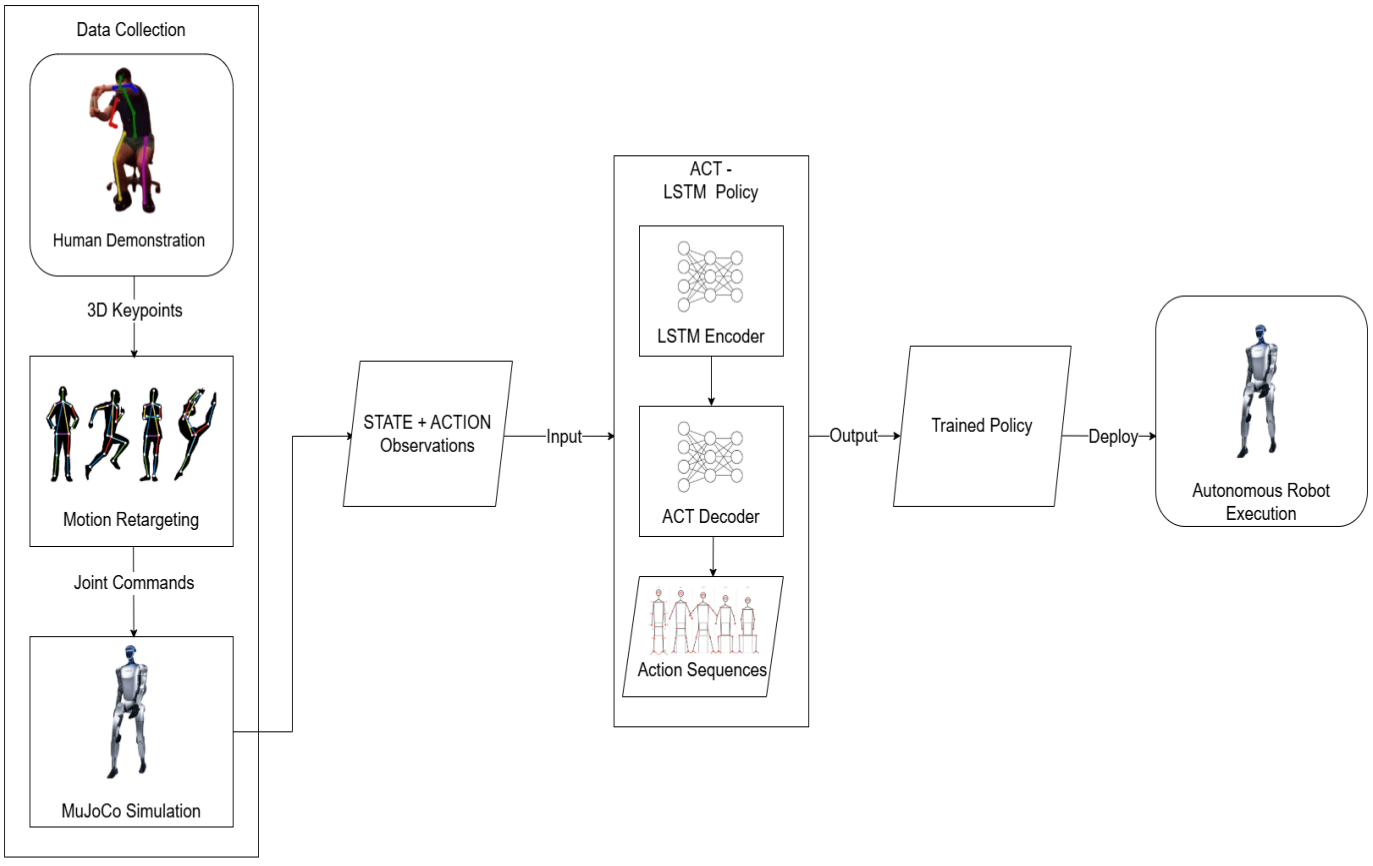

Figure 3.1. Conceptual framework of the proposed state-based imitation learning pipeline for bimanual box pickup on the Unitree G1 humanoid robot.

3.3 Demonstration Collection

The demonstration collection stage is the first phase of the proposed pipeline and serves as the primary source of training data for the ACT-LSTM imitation learning policy. To record the full-body skeletal movement of the human demonstrator in real time, a Zed stereo depth camera will record the motion of the human demonstrator, with two parallel motion retargeting strategies, a position-based approach to the arms through inverse kinematics, and a velocity-based approach to the legs through a pre-trained walking policy in the unitree_rl_gym repository. The result of this step will be a dataset of pairs of state-action trajectories as observed in the MuJoCo simulation environment.

3.3.1 Scene Setup in MuJoCo

Before every demonstration episode, the simulation environment will be set up to specify the task context. The Unitree G1 humanoid robot model will be loaded based on the MuJoCo Menagerie repository with the g1_29dof_rev_1_0 configuration that will enable 29 actuated joints in the legs, the waist, and the arms. The robot will be started in an upright neutral standing posture at a fixed starting point in the simulation world, all the joints of the robot in the zero reference configuration.

Two fixed rectangular platforms that are to be modeled as fixed rigid bodies in MuJoCo will be used in the scene, with each platform having a surface height of 0.75 m above the ground plane. Each episode will start with a box object of a fixed size and fixed orientation spawned at a randomized position on the surface of the first platform. Sampling of box positions will be done in an even manner over the surface of the platform in the horizontal plane where the training data will have a wide variety of spatial configurations. A goal location will be defined as a fixed marked region on the surface of the second platform, where the robot is expected to place the box at the end of each episode. Since both platforms and the goal location remain fixed across all episodes, their positions are not included in the policy state observation.

The simulation physics will be configured with an internal integration timestep of 0.002 seconds corresponding to 500 Hz, with joint commands sent at a control frequency of 25 Hz. Gravitational acceleration will be set to $g = 9.8, m/s^2$. These parameters are consistent with standard practice for humanoid robot simulation in MuJoCo (Todorov et al., 2012). Table 3.1 summarizes the complete simulation configuration for the demonstration collection stage.

Table 3.1: Simulation Configuration for Demonstration Collection

Parameter	Value
Robot model	g1_29dof_rev_1_0
Physics timestep	0.002 s (500 Hz)
Control frequency	25 Hz
Gravitational acceleration	9.8 m/s²
Robot initialization	Upright standing, all joints at zero
Box size	Fixed
Box orientation	Fixed
Box position	Randomized per episode
Platform height	0.75 m
Number of platforms	2
Goal location	Fixed marked region on platform 2

3.3.2 Zed Camera Setup and Body Tracking

The Zed stereo depth camera will be positioned in front of the human demonstrator at approximately chest height, at a distance of 1.5 to 2 meters, allowing both arms and the pelvis to remain fully visible throughout the demonstration. The

Zed SDK's body tracking module will extract 3D keypoint positions of the human demonstrator's body at each camera frame, operating at approximately 30 Hz.

The keypoints extracted by the Zed SDK will be organized into two groups according to their function in the pipeline. The arm tracking group includes both wrists, both elbows, and both shoulders, which will serve as the basis for computing end-effector targets for the robot's bimanual arm control. The locomotion group includes the pelvis center, which will serve as the reference point for computing the demonstrator's walking velocity and direction.

3.3.3 Coordinate Transformation

The 3D keypoint positions extracted by the Zed SDK are expressed in the camera reference frame, which has its origin at the RGB-D camera, with the X-axis pointing to the right of the image plane, the Y-axis pointing downward, and the Z-axis pointing forward along the camera optical axis. To be used as end-effector targets and locomotion inputs in MuJoCo, these keypoint coordinates must be transformed into the robot-aligned reference frame, which has its origin at the center of the robot's hips, with the X-axis pointing forward, the Y-axis pointing to the left, and the Z-axis pointing upward.

Following the coordinate transformation approach validated by Suárez Martínez (2026) on the same Unitree G1 platform, a fixed rotation matrix R is applied to each extracted keypoint position to map it from the camera frame to the robot-aligned frame. Let $p_k^{cam} = [x, y, z]^T$ denote the position of keypoint k in the camera reference frame. The rotation transformation is used to obtain the corresponding position in the robot-aligned reference frame:

$$p_k^{rob} = R\, p_k^{cam} \qquad \text{(Equation 3.1)}$$

The correspondence of the axis between the camera frame and the robot-aligned frame is determined as:

$$X_{rob} = Z_{cam}, \quad Y_{rob} = -X_{cam}, \quad Z_{rob} = -Y_{cam}$$

The result of this correspondence is the following rotation matrix:

$$R = \begin{bmatrix} 0 & 0 & 1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$

This transformation will make the assumption that the camera optical axis is roughly parallel to the direction of the demonstrator, which will be fulfilled by the physical experimental setup. When this transformation is applied to all the extracted keypoints, it now gives 3D positions that are in the robot-aligned reference frame centered at the hip joint, which serves as the geometric inputs to the arm teleoperation and the walking detection stages described in the subsequent subsections.

3.3.4 Walking Detection and Locomotion Triggering

The walking detection part will track the position of the keypoint of the pelvis throughout the time to deduce the locomotion intent of the demonstrator and convert it into the velocity command to the walking controller of the robot. The pelvis center is the center of reference to detect locomotion since the translational movement of the pelvis directly relates to the direction and the speed of bipedal walking.

The Zed SDK gives the position of the pelvis center at every camera frame in the robot-aligned reference frame. Translational velocity of the pelvis center is determined by calculating the extent to which the pelvis has traveled in a brief sliding window over the recent frames in comparison to the time passed. This gives a velocity vector of the speed and direction in which the demonstrator is walking.

Once the size of this velocity surpasses a specified limit, the system produces a walking command with the forward speed, lateral speed, and turning rate that are sent to the trained walking policy in the unitree_rl_gym repository. When the pelvis velocity falls below the threshold, a zero velocity command is sent to stop the robots walking. There is a minimum duration requirement that the walking command is activated and that short-term postural changes or weight transfer, so that it does not cause unintended walking. Preliminary calibration will be done to determine all threshold and timing values before demonstration collection.

These velocity commands are accepted by the pre-trained walking policy of unitree_rl_gym and result in stable bipedal locomotion in MuJoCo simulation, solely controlling the leg joints of the robot. The arm teleoperation module uses the arm joints simultaneously and independently, this means that the robot can walk and reach at the same time without a conflict of control streams.

It is acknowledged that the minimum duration requirement and velocity estimation window creates an internal delay between when a human is walking and when the robot responds to locomotion. This latency will be defined in the initial calibration and countered by demonstrator training where demonstrators will be trained to wait a short time before beginning to walk so that the detection system can be activated before the pelvis has moved significantly.

3.3.5 Arm Teleoperation via Geometric Scaling and Inverse Kinematics

The arm teleoperation subsystem will convert the arm keypoints of the demonstrator into joint commands of the robot arm with the geometry scaling and inverse kinematics method validated by Suarez Martinez (2026).

Geometric Scaling

The length of the arm segments of the human demonstrator is not the same as the length of the arm segments of the Unitree G1. Geometric scaling addresses this by preserving the spatial direction of each human arm segment while rescaling its length to match the corresponding robot link dimension. For each arm, let p_{sh}, p_{el}, and p_{wr} denote the shoulder, elbow, and wrist keypoint positions of the human demonstrator expressed in the robot-aligned reference frame. The unit direction vectors of the upper arm and forearm segments are computed as:

$$\hat{u}_{ua} = \frac{p_{el} - p_{sh}}{\|p_{el} - p_{sh}\|} \quad \text{(Equation 3.2)}$$

$$\hat{u}_{fa} = \frac{p_{wr} - p_{el}}{\|p_{wr} - p_{el}\|} \quad \text{(Equation 3.3)}$$

Let L_{ua} and L_{fa} denote the upper arm and forearm lengths of the Unitree G1 measured from its MJCF model file. The scaled target elbow and wrist positions are computed as:

$$p_{el}^{rob} = p_{sh}^{rob} + L_{ua} \cdot \hat{u}_{ua} \quad \text{(Equation 3.4)}$$

$$p_{wr}^{rob} = p_{el}^{rob} + L_{fa} \cdot \hat{u}_{fa} \quad \text{(Equation 3.5)}$$

where p_{sh}^{rob} is the shoulder position of the Unitree G1 obtained from its MJCF model file. This procedure is applied independently to both the left and right arms.

Inverse Kinematics

The scaled target positions are passed to MuJoCo's numerical inverse kinematics solver to compute the joint angles required to place the robot's

end-effectors at the desired locations. Let $q \in \mathbb{R}^{14}$ denote the vector of joint angles for both arms combined. The inverse kinematics problem is formulated as:

$$er = \min_{q} \sum_{i} \left\| p_i(q) - p_i^{target} \right\|$$

(Equation 3.6)

where $p_i(q)$ is the current end-effector position computed through forward kinematics and p_i^{target} is the scaled target position. The solver iteratively updates q to reduce this error until convergence.

Joint Limit Enforcement and Temporal Smoothing

The raw joint angles computed by the IK solver are constrained to the mechanical range of each joint as defined in the Unitree G1 MJCF model file:

$$q_k^{clipped} = \max\left(q_k^{min},\ \min\left(q_k^{max},\ q_k\right)\right)$$

(Equation 3.7)

A first-order low-pass filter is then applied to remove frame-level jitter, following the smoothing approach of Suárez Martínez (2026):

$$q_k^{smooth} = (1 - \alpha) \cdot q_{k,prev}^{smooth} + \alpha \cdot q_k^{clipped}$$

(Equation 3.8)

where $\alpha = 0.8$ is the smoothing factor. The smoothed joint angles are sent to the MuJoCo arm actuators at 25 Hz.

3.3.6 Bimanual Grasping Trigger

During demonstration collection, gripper commands will be triggered by tracking a target position defined slightly inside each face of the box relative to its center. The left gripper target will be positioned slightly past the left face of the box toward its center, and the right gripper target will be positioned slightly past the right face of the box toward its center. These target positions are computed from the box center position read directly from MuJoCo's internal state at each timestep.

By commanding the grippers toward positions inside the box surface, the inverse kinematics solver continuously drives the grippers inward against the box faces. Since the box physically prevents the grippers from reaching their targets, a sustained contact force is maintained pressing inward on both sides of the box simultaneously. This internal pushing creates enough friction between the bodies of the grippers and the surfaces of the boxes to ensure that the grippers tightly hold the box during the pickup and transportation stages of the task.

The gripper target positions are updated at each simulation timestep, depending on the current box position. This makes sure the force of inward pressing is kept constant as the robot walks with the box. The particular inward offset distance will be established in the initial implementation to guarantee dependable grasping throughout the variety of box positions employed in the demonstration dataset. The gripper command is a binary scalar where zero corresponds to open and one corresponds to close, and is logged at each timestep as a component of the full action vector in Section 3.4.

3.3.8 Demonstration Protocol

A demonstration of approximately 100-150 episodes will comprise the training dataset. Each episode will start the box spawned at a new randomized location on the surface of the first platform, sampled according to a uniform distribution over the surface of the platform. The human demonstrator will perform the complete task sequence in front of the Zed camera: walk toward the first platform, reach forward with both arms and close both hands to pick up the box, carry the box by walking toward the second platform, and lower both arms and open both hands to place the box at the goal region.

Prior to data collection, the demonstrator's physical walking space will be calibrated to ensure that pelvis displacement in the real environment corresponds proportionally to robot displacement in MuJoCo. Box positions will be varied across

episodes with no two consecutive episodes sharing the same configuration, ensuring that the demonstration dataset covers a wide range of spatial positions on the platform surface. For evaluation, out-of-distribution testing will be based on box positions, which will be sampled based on locations that are not present in any demonstration episode, such that Experiment 2 is an evaluation of how the trained policy behaves in locations never seen during training.

Incidents where the robot fails to grasp the box, drops it on the way to the location of the task or creates an incomplete task trajectory will be removed and re-recorded. One episode will be visually examined at the time of collection to detect and filter out low-quality demonstrations prior to their inclusion in the training dataset. Table 3.2 summarizes the demonstration collection protocol.

Table 3.2: Demonstration Collection Protocol

Parameter	Value
Total demonstrations	100 to 150 episodes
Box spawn position	Randomized on platform 1 training region
Box size	Fixed
Box orientation	Fixed
Walking demonstration type	Walking in space
Logging frequency	25 Hz
Episode rejection criterion	Incomplete task sequence
Quality control	Visual inspection per episode

3.4 State and Action Representation

The policy used in this experiment runs on a privileged simulator state, where all observations are accessed at the internal data structures of MuJoCo at each timestep. This section specifies the actual structure of the state vector and action vector that will be used in the pipeline.

3.4.1 State Vector

The state vector $s_t \in \mathbb{R}^S$ encodes the complete observable configuration of the robot and the task environment at simulation timestep t. It is composed of six groups of variables, each capturing a distinct aspect of the task.

Box Pose. The position and orientation of the box in the simulation world frame constitute the primary task-relevant variables. Box position is represented as $p_{box} \in \mathbb{R}^3$ and box orientation as a unit quaternion $q_{box} \in \mathbb{R}^4$.

Robot Base Pose. Since the robot locomotes during the task, the policy requires the robot's current position and orientation in the world frame. The robot base position is denoted $p_{base} \in \mathbb{R}^3$ and the robot base orientation as a unit quaternion $q_{base} \in \mathbb{R}^4$.

End-Effector Poses. The positions and orientations of the left and right grippers in the world frame provide the policy with complete information about where the robot's hands are and how they are oriented at each timestep. The left gripper position is denoted $p_{grip,L} \in \mathbb{R}^3$, the left gripper orientation as $q_{grip,L} \in \mathbb{R}^4$, the right gripper position as $p_{grip,R} \in \mathbb{R}^3$, and the right gripper orientation as $q_{grip,R} \in \mathbb{R}^4$.

Gripper States. The open or closed state of each gripper indicates whether the robot is currently holding the box, represented as continuous scalars $g_L \in [0, 1]$ and $g_R \in [0, 1]$, where zero indicates fully open and one indicates fully closed.

Arm and Waist Joint Angles. The joint angles of both arms and the waist provide the policy with proprioceptive information about the robot's current upper body configuration. The left arm joint angles are denoted $\theta_L \in \mathbb{R}^7$, the right arm joint angles as $\theta_R \in \mathbb{R}^7$, and the waist joint angles as $\theta_{waist} \in \mathbb{R}^3$.

This yields a total state dimensionality of:

$$s_t = \left[p_{box}^T,\ q_{box}^T,\ p_{base}^T,\ q_{base}^T,\ p_{grip,L}^T,\ q_{grip,L}^T,\ p_{grip,R}^T,\ q_{grip,R}^T,\ g_L,\ g_R,\ \theta_L^T,\ \theta_R^T,\ \theta_{waist}^T\right]^T$$

$$S = 3 + 4 + 3 + 4 + 3 + 4 + 3 + 4 + 1 + 1 + 7 + 7 + 3 = 47$$

Table 3.3 summarizes the complete composition of the state vector.

Table 3.3: State Vector Composition

Variable	Symbol	Dimensions	Source
Box position	p_{box}	3	MuJoCo object state
Box orientation	q_{box}	4	MuJoCo object state
Robot base position	p_{base}	3	MuJoCo body state
Robot base orientation	q_{base}	4	MuJoCo body state
Left gripper position	$p_{grip,L}$	3	MuJoCo site position
Left gripper orientation	$q_{grip,L}$	4	MuJoCo site orientation
Right gripper position	$p_{grip,R}$	3	MuJoCo site position

Right gripper orientation	$q_{grip,R}$	4	MuJoCo site orientation
Left gripper state	g_L	1	MuJoCo actuator state
Right gripper state	g_R	1	MuJoCo actuator state
Left arm joint angles	θ_L	7	MuJoCo joint positions
Right arm joint angles	θ_R	7	MuJoCo joint positions
Waist joint angles	θ_{waist}	3	MuJoCo joint positions
Total		47	

The leg joint angles of the Unitree G1 are excluded from the state vector. Leg motion is controlled throughout each episode by the pre-trained walking policy from unitree_rl_gym and is not part of the behavior the ACT-LSTM policy will learn.

3.4.2 Action Vector

The action vector $a_t \in \mathbb{R}^A$ defines the commands the policy outputs at each timestep. The policy controls the robot's arms, waist, grippers, and locomotion, producing a complete loco-manipulation behavior learned entirely from human demonstrations.

Arm and Waist Joint Targets. The policy outputs target joint positions for all fourteen arm joints and three waist joints, denoted $a_{\theta,L} \in \mathbb{R}^7$, $a_{\theta,R} \in \mathbb{R}^7$, and $a_{\theta,waist} \in \mathbb{R}^3$ respectively.

Gripper Commands. The policy outputs a continuous gripper command for each hand, denoted $a_{g,L} \in [0, 1]$ and $a_{g,R} \in [0, 1]$, where zero corresponds to fully open and one to fully closed.

Walking Velocity Command. The policy outputs a three-dimensional walking velocity command $a_{walk} \in \mathbb{R}^3$ specifying the desired forward speed, lateral speed, and turning rate, which is passed directly to the pre-trained walking policy from unitree_rl_gym (Unitree Robotics, 2024) at each timestep.

The total action dimensionality is:

$$a_t = \left[a_{\theta,L}^T,\ a_{\theta,R}^T,\ a_{\theta,waist}^T,\ a_{g,L},\ a_{g,R},\ a_{walk}^T\right]^T$$

$$A = 7 + 7 + 3 + 1 + 1 + 3 = 22$$

Table 3.4 summarizes the complete composition of the action vector.

Table 3.4: Action Vector Composition

Variable	Symbol	Dimensions	Description
Left arm joint targets	$a_{\theta,L}$	7	Target joint positions for left arm
Right arm joint targets	$a_{\theta,R}$	7	Target joint positions for right arm
Waist joint targets	$a_{\theta,waist}$	3	Target joint positions for waist
Left gripper command	$a_{g,L}$	1	Desired open/closed state, left gripper
Right gripper command	$a_{g,R}$	1	Desired open/closed state, right gripper
Walking velocity command	a_{walk}	3	Forward speed, lateral speed, turning rate
Total		**22**	

3.5 Data Preprocessing

Prior to training, the collected demonstration dataset will be processed into a format compatible with the ACT-LSTM training pipeline. This stage consists of three steps: dataset splitting, normalization, and action chunk construction.

Dataset Splitting. The collected demonstration episodes will be divided into a training set and a validation set using an 80/20 split, where 80 percent of episodes are used for training and 20 percent are reserved for validation. The split will be stratified across box positions to ensure that both sets contain a representative range of spatial configurations. At the end of each training epoch, the same L1 and KL divergence loss function defined in Section 3.7 will be computed on the validation set without updating the model weights, allowing training progress to be monitored on held-out demonstration data. Training will be automatically terminated when 10 consecutive epochs show no improvement in validation loss, at which point the model checkpoint with the lowest validation loss will be saved for deployment. A separate set of evaluation episodes, distinct from both the training and validation sets, will be used for autonomous policy evaluation as described in Section 3.8.

Normalization. Both the state and action vectors are normalized per dimension using the mean and standard deviation computed from the training set only. These normalization statistics are applied consistently during validation, testing, and autonomous deployment.

Action Chunk Construction. The ACT policy is trained to predict sequences of future actions rather than single actions at each timestep, which reduces the effective task horizon and mitigates compounding errors in behavioral cloning (Zhao et al., 2023). For each timestep t in a demonstration episode, an observation-chunk pair is constructed consisting of a context window of the last W_o state observations and a corresponding target chunk of the next K actions :

$$\text{input}_t = [s_{t-W_o+1},\ s_{t-W_o+2},\ \ldots,\ s_t]$$

$$\text{target}_t = [a_{t+1},\ a_{t+2},\ \ldots,\ a_{t+K}]$$

where W_o is the observation window size and K is the action chunk size. Pairs where the target chunk extends beyond the end of the episode are discarded.

3.6 ACT-LSTM Policy Architecture

The proposed pipeline trains an imitation learning policy using a combined Action Chunking with Transformers and Long Short-Term Memory (ACT-LSTM) architecture. The architecture consists of two components: an LSTM encoder that encodes temporal context from recent state history, and an action chunk decoder that predicts sequences of future actions at each timestep.

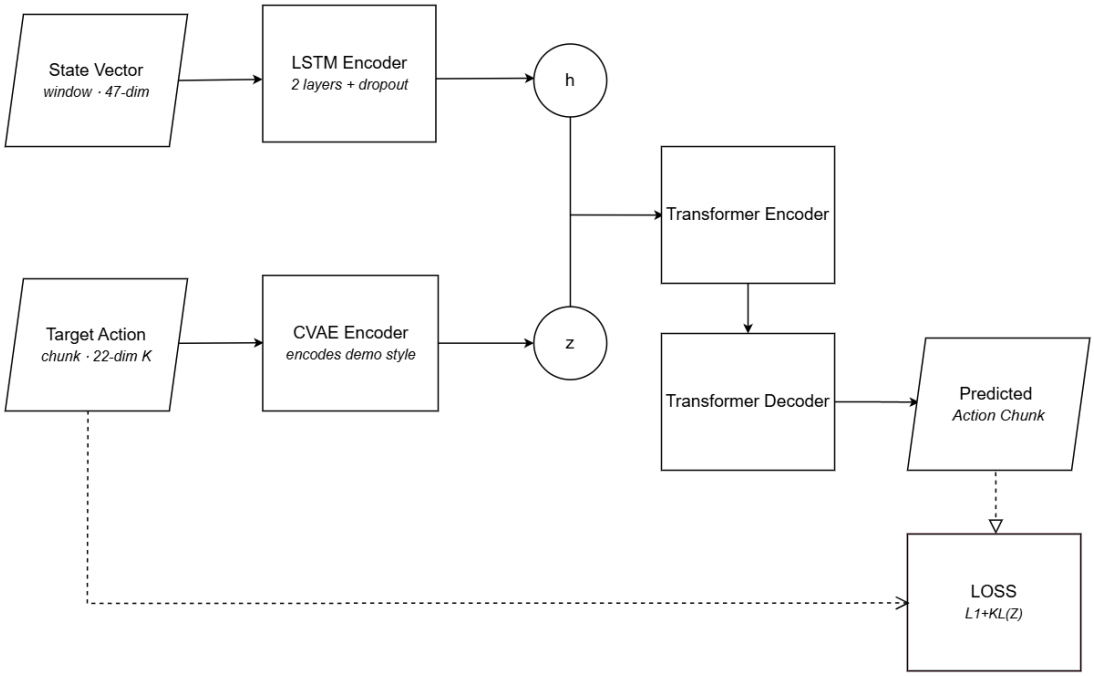

Figure 3.2. ACT-LSTM policy architecture.

LSTM Encoder. The LSTM encoder processes a rolling window of the last Wo normalized state observations and produces a hidden state vector h summarizing the recent task history. This allows the policy to distinguish between the sequential

phases of the loco-manipulation task based on the pattern of recent states rather than the current state alone. The sequential phases include the walking approach, bimanual pickup, box transport, and box placement. Two stacked LSTM layers are used with a dropout rate of 0.3 applied between layers during training.

Action Chunk Decoder. The action chunk decoder receives the current state observation s□ and the LSTM hidden state h□ as input and predicts the next K actions as a sequence. Both s□ and h□ are projected and concatenated as input tokens to a transformer encoder, which synthesizes this information and passes it to a transformer decoder to generate the predicted action chunk. The decoder is trained using a Conditional Variational Autoencoder objective. During training, a CVAE encoder compresses the current state observation and the ground truth action sequence into a style variable z that captures variability across demonstrations. The CVAE encoder is used only during training and is discarded during deployment, where z is set to zero for deterministic prediction. During deployment, temporal ensembling is applied where overlapping action chunk predictions are combined through an exponential weighting scheme to produce smoother robot motion (Zhao et al., 2023).

The LSTM encoder and action chunk decoder are trained jointly end-to-end through behavioral cloning on the collected demonstration dataset. The training objective minimizes the L1 reconstruction loss between the predicted and ground truth action chunks, combined with a KL divergence regularization term on the style variable z. Specific hyperparameter values including the LSTM hidden dimension, action chunk size, observation window size, and regularization weight will be determined during the implementation and training phase.

To serve as a baseline for comparison, a standard behavioral cloning policy will also be implemented. The BC baseline consists of a feedforward neural network that receives the current state observation s_t as input and directly predicts a single action a_t at each timestep, without action chunking or recurrent memory. Unlike the

ACT-LSTM policy, the BC baseline makes no assumptions about temporal structure and represents the simplest possible imitation learning approach for this task.

3.7 Training Procedure

The ACT-LSTM policy and the behavioral cloning baseline will each be trained through behavioral cloning on the same demonstration dataset using identical training procedures and hyperparameters to ensure a fair comparison.

Loss Function. The training objective minimizes the L1 reconstruction loss between the predicted action chunk and the ground truth action chunk from the demonstration data, combined with a KL divergence regularization term on the style variable z (Zhao et al., 2023). L1 loss is used rather than L2 loss, as Zhao et al. (2023) noted that L1 leads to more precise modeling of action sequences. The L1 reconstruction loss is defined as:

$$L_1 = \frac{1}{K} \sum_{k=1}^{K} |\hat{a}_k - a_k| \qquad \text{(Equation 3.9)}$$

where K is the number of steps in the action chunk, $\hat{a}_k$ is the predicted action at step k, and a_k is the ground truth action at step k The KL divergence term regularizes the CVAE encoder to a standard Gaussian prior, with a regularization hyperparameter β that will be set during the implementation stage. The combined training objective is:

$$\mathcal{L} = L_1(\hat{a}, a) + \beta \cdot D_{KL}\left(q_\phi(z \mid s, a) \,\|\, \mathcal{N}(0, I)\right)$$

(Equation 3.10)

For the BC baseline, only the L1 reconstruction loss is used since the BC policy does not include a CVAE encoder or style variable z.

Optimizer and Learning Rate. The model will be trained using the Adam optimizer and with the initial learning rate of 1×10^{-5}. During training, a cosine annealing learning rate schedule will be used, which will slowly decrease the learning rate and allow the optimizer to approach a more optimal solution.

Training Configuration. Training will run for a maximum number of epochs to be determined during the implementation phase, based on the convergence behavior observed on the demonstration dataset. The batch size will be set following the training configuration of Zhao et al. (2023) as a starting point and adjusted based on available GPU memory during implementation. Training will be automatically terminated when 10 consecutive epochs show no improvement in validation loss, at which point the model checkpoint with the lowest validation loss will be saved for deployment. All training will be conducted on the university GPU facility.

3.8 Simulation and Evaluation

The trained ACT-LSTM policy will be evaluated entirely within the MuJoCo physics simulation environment using the Unitree G1 humanoid robot model. All evaluation episodes will use privileged simulator state as policy input, consistent with the state representation defined in Section 3.4. This section describes the simulation setup, autonomous deployment procedure, evaluation metrics, and experimental conditions used to answer the research questions of this study.

3.8.1 Simulation Setup

The testing setup will involve the same MuJoCo scene setup described in Section 3.3.1, including two stationary platforms, the Unitree G1 model, and the box object. At the beginning of every evaluation episode, the robot will be started in its fixed starting position with all the joints in the zero reference configuration. The box will be spawned in a randomized location on the platform surface based on the spatial split described in Section 3.3.8, with in-distribution locations are sampled in

the training region and out-of-distribution locations are sampled in the held-out test region.

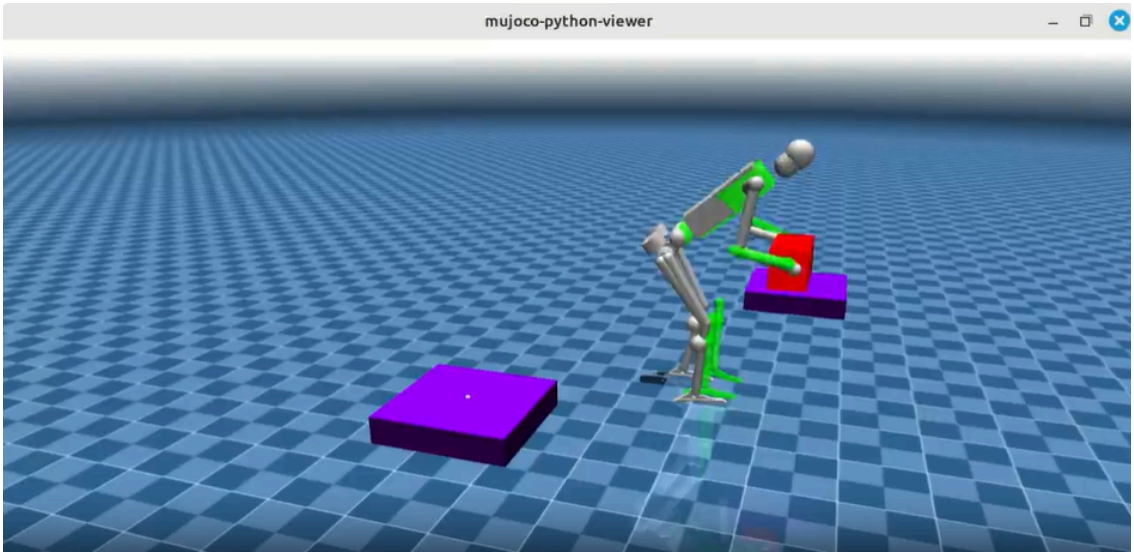

Figure 3.3. MuJoCo Environment Setup. Adapted from Singh et al. (2024).

3.8.2 Autonomous Deployment

During evaluation, the trained ACT-LSTM policy will operate autonomously without any human input. At each simulation timestep, the policy will receive the current state observation from MuJoCo, produce an action chunk through the forward pass described in Section 3.6, and apply the first action of the chunk to the robot's actuators. The walking velocity component of the action vector will be passed to the pre-trained walking policy from unitree_rl_gym, which will produce the corresponding leg joint commands. The arm and waist joint targets and gripper commands will be applied directly to the robot's position-controlled actuators. This process will repeat at each timestep until the episode terminates.

Task performance will be evaluated across four sequential phase-level criteria, each assessed independently throughout and at the end of each episode. Grasping success is determined when both gripper states register as closed and the box height rises above the platform surface, confirming that the robot secured the box. Lifting success is determined when the box height exceeds the platform surface height by at

least 0.05m, confirming that the box was lifted clear of the platform. Transport success is determined when the box remains above floor level throughout the walking phase without being dropped. Placement success is determined when the Euclidean distance between the final box position and the center of the goal region falls below a defined placement threshold d_place:

$$\|\mathbf{p}_{box} - \mathbf{p}_{goal}\| < d_{place}$$

where p_{box} is the final box position and p_{goal} is the center of the goal region. The value of d_{place} will be set during preliminary implementation. An episode will be terminated as a failure if the robot drops the box during transport, fails to grasp the box within a defined number of timesteps, or exceeds the maximum episode length of 500 timesteps.

3.8.3 Evaluation Metrics

The performance of both the ACT-LSTM policy and the behavioral cloning baseline will be assessed using two quantitative metrics.

Task Success Rate. Task performance for both policies will be reported as four independent phase-level success rates, each computed as the percentage of evaluation episodes in which the robot successfully completes the corresponding phase:

$$\text{Success Rate} = \frac{\text{Number of successful episodes}}{\text{Total number of episodes}} \times 100\%$$

(Equation 3.11)

The four phase-level success rates reported are grasping success rate, lifting success rate, transport success rate, and placement success rate. Reporting success rates at each phase provides a detailed picture of where each policy performs well and where it falls short, directly answering the research questions by quantifying

how effectively each policy executes the task and how well it generalizes to unseen box positions.

Generalization Gap. The generalization capability of the each policy will be quantified by computing the difference in phase-level success rates between Experiment 1 and Experiment 2:

$$\Delta S_{phase} = S_{in} - S_{out} \qquad \text{(Equation 3.12)}$$

where S_{in} is the phase-level success rate on in-distribution box positions from Experiment 1 and S_{out} is the phase-level success rate on out-of-distribution box positions from Experiment 2. A smaller ΔS_{phase} indicates better generalization to unseen box positions. This metric is computed for both the ACT-LSTM policy and the behavioral cloning baseline, directly answering RQ3 by quantifying the extent to which each policy maintains its performance on box positions not encountered during demonstration collection.

3.8.4 Evaluation Experiments

Two evaluation experiments will be conducted, each corresponding to one research question of this study.

Experiment 1 - In-Distribution Task Execution

This experiment will evaluate both the ACT-LSTM policy and the behavioral cloning baseline on 100 episodes with box positions encountered during demonstration collection. This experiment directly answers RQ1 and RQ2 by measuring how effectively each policy executes bimanual box pickup and placement on familiar configurations, as measured by the four phase-level success rates defined in Section 3.8.3.

Experiment 2 - Out-of-Distribution Generalization

This experiment will evaluate both the ACT-LSTM policy and the behavioral cloning baseline on 100 episodes with box positions not encountered during any demonstration episode. This experiment directly answers RQ3 by measuring the extent to which each policy generalizes its learned behavior to unseen box positions. The phase-level success rates from Experiment 2 will be compared against Experiment 1 to quantify the generalization gap for each policy across all four phases of the task.

3.9 Hardware and Software

All experiments will be conducted using the hardware and software specifications described in this section.

Hardware. Demonstration collection will use a Zed stereo depth camera for human body tracking. Policy training will be conducted on the university GPU facility, which provides sufficient computational resources for training the ACT-LSTM model within a reasonable timeframe. Local development and testing will be performed on the researchers' personal computers. Table 3.5 summarizes the hardware specifications used in this study.

Table 3.5: Hardware Specifications

Component	Specification
Demonstration sensor	Zed stereo depth camera
Training hardware	University GPU facility
Local development CPU	Intel Core i5-11400H, 2.70 GHz
Local development GPU	NVIDIA GeForce RTX 3050 Laptop, 4GB VRAM
Local development RAM	8GB DDR4
Operating system	Ubuntu 22.04 LTS

Software. The proposed pipeline will be implemented in Python. MuJoCo 3.7.0 will serve as the physics simulation environment for both demonstration collection and policy evaluation. The ZED SDK will be used for real-time body tracking during demonstration collection. The unitree_rl_gym repository will provide the pre-trained walking policy for locomotion control. PyTorch will serve as the deep learning framework for implementing and training the ACT-LSTM policy. ROS2 Jazzy may be adopted as middleware for inter-module communication following the system architecture of Suárez Martínez (2026), subject to evaluation during the implementation phase. Supporting libraries including NumPy and SciPy will be used for numerical computation and data processing. Table 3.6 summarizes the software stack used in this study.

Table 3.6: Software Stack

Component	Tool
Physics simulation	MuJoCo 3.7.0
Body tracking	ZED SDK
Walking policy	unitree_rl_gym
Deep learning framework	PyTorch
Middleware	ROS2 Jazzy
Numerical computation	NumPy, SciPy
Programming language	Python

REFERENCES

Baek, S., Kim, A., Choi, J., Ha, E., & Kim, J. (2024). Human motion retargeting to a full-scale humanoid robot using a monocular camera and human pose estimation. *International Journal of Control Automation and Systems, 22*(9), 2860–2870. https://doi.org/10.1007/s12555-023-0686-y

Chen, M.-X., Firat, O., Bapna, A., Johnson, M., Macherey, W., Foster, G., Jones, L., Parmar, N., Schuster, M., Chen, Z., Wu, Y., & Hughes, M. (2018). The best of both worlds: Combining recent advances in neural machine translation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 76–86). Association for Computational Linguistics. https://doi.org/10.18653/v1/P18-1008

Chi, C., Xu, Z., Pan, C., Cousineau, E., Burchfiel, B., Feng, S., Tedrake, R., & Song, S. (2024). Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots. *arXiv preprint arXiv:2402.10329*. https://arxiv.org/abs/2402.10329

DeepMind. (2022). *MuJoCo* [Software]. GitHub. https://github.com/google-deepmind/mujoco

Fu, Z., Zhao, Q., Wu, Q., Wetzstein, G., & Finn, C. (2024). HumanPlus: Humanoid shadowing and imitation from humans. *arXiv preprint arXiv:2406.10454*. https://arxiv.org/abs/2406.10454

Grannen, J., Wu, Y., Vu, B., & Sadigh, D. (2023). Stabilize to act: Learning to coordinate for bimanual manipulation. In J. Tan, M. Toussaint, & K. Darvish (Eds.), Proceedings of the 7th Conference on Robot Learning (Vol. 229, pp. 563–576). Proceedings of Machine Learning Research. https://proceedings.mlr.press/v229/grannen23a.html

Graßhof, S., Bastholm, M., & Brandt, S. S. (2023). Neural network-based human motion predictor and smoother. *SN Computer Science, 4*(6). https://doi.org/10.1007/s42979-023-02195-0

Graves, A., Mohamed, A., & Hinton, G. (2013). Speech recognition with deep recurrent neural networks. *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*, 6645–6649. https://doi.org/10.1109/ICASSP.2013.6638947

Hartley, R., & Zisserman, A. (2004). *Multiple view geometry in computer vision* (2nd ed.). Cambridge University Press. https://doi.org/10.1017/CBO9780511811685

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Hussein, A., Gaber, M. M., Elyan, E., & Jayne, C. (2017). Imitation learning: A survey of learning methods. *ACM Computing Surveys, 50*(2), 1–35. https://doi.org/10.1145/3054912

Jang, E., Irpan, A., Khansari, M., Kappler, D., Ebert, F., Lynch, C., Levine, S., & Finn, C. (2022). BC-Z: Zero-shot task generalization with robotic imitation learning. In *Proceedings of the 5th Conference on Robot Learning (CoRL 2021), Proceedings of Machine Learning Research, 164*, 991–1002. PMLR. https://proceedings.mlr.press/v164/jang22a.html

Kuindersma, S., Deits, R., Fallon, M., Valenzuela, A., Dai, H., Permenter, F., Koolen, T., Marion, P., & Tedrake, R. (2016). Optimization-based locomotion planning, estimation, and control design for the Atlas humanoid robot. *Autonomous Robots, 40*(3), 429–455. https://doi.org/10.1007/s10514-015-9479-3

Mandlekar, A., Xu, D., Wong, J., Nasiriany, S., Wang, C., Kulkarni, R., Fei-Fei, L., Savarese, S., Zhu, Y., & Martín-Martín, R. (2021). What matters in learning from offline human demonstrations for robot manipulation. In *Proceedings of the*

5th Conference on Robot Learning (Vol. 164, pp. 1678–1690). PMLR. https://proceedings.mlr.press/v164/mandlekar22a.html

Mark, M. S., Liang, J., Attarian, M., Fu, C., Dwibedi, D., Shah, D., & Kumar, A. (2026). *BPP: Long-context robot imitation learning by focusing on key history frames* [Preprint]. arXiv. https://arxiv.org/abs/2602.15010

Muratore, F., Ramos, F., Turk, G., Yu, W., Gienger, M., & Peters, J. (2022). Robot learning from randomized simulations: A review. *Frontiers in Robotics and AI, 9*, 799893. https://doi.org/10.3389/frobt.2022.799893

Pirk, S., Hausman, K., Toshev, A., & Khansari, M. (2021). Modeling long-horizon tasks as sequential interaction landscapes. In *Proceedings of the 2020 Conference on Robot Learning* (Vol. 155, pp. 471–484). PMLR. https://proceedings.mlr.press/v155/pirk21a.html

Rahmatizadeh, R., Abolghasemi, P., Bölöni, L., & Levine, S. (2018). Vision-based multi-task manipulation for inexpensive robots using end-to-end learning from demonstration. In *2018 IEEE International Conference on Robotics and Automation (ICRA)* (pp. 3758–3765). IEEE. https://doi.org/10.1109/ICRA.2018.8461076

Ross, S., & Bagnell, D. (2010). Efficient reductions for imitation learning. *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 661–668. http://proceedings.mlr.press/v9/ross10a.html

Ross, S., Gordon, G., & Bagnell, D. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 627–635. http://proceedings.mlr.press/v15/ross11a.html

Seo, M., Han, S., Sim, K., Bang, S. H., Gonzalez, C., Sentis, L., & Zhu, Y. (2023). Deep imitation learning for humanoid loco-manipulation through human teleoperation. https://arxiv.org/abs/2309.01952

Siciliano, B., Sciavicco, L., Villani, L., & Oriolo, G. (2009). *Robotics: Modelling, planning and control*. Springer. https://doi.org/10.1007/978-1-84628-642-1

Singh, R. P., Gergondet, P., & Kanehiro, F. (2023). Mc-MuJoCo: Simulating articulated robots with FSM controllers in MuJoCo. *2023 IEEE/SICE International Symposium on System Integration*, 1–5. https://doi.org/10.1109/SII55687.2023.10039218

Singh, R. P., Leziart, P.-A., Murooka, M., Morisawa, M., Yoshida, E., & Kanehiro, F. (2024). Expert-guided imitation for learning humanoid loco-manipulation from motion capture. https://paleziart.github.io/guided-humanoid-locomanipulation/

Sohn, K., Lee, H., & Yan, X. (2015). Learning structured output representation using deep conditional generative models. In *Advances in Neural Information Processing Systems 28* (pp. 3483–3491). Curran Associates. https://proceedings.neurips.cc/paper/2015/hash/8d55a249e6baa5c06772297520da2051-Abstract.html

StereoLabs. (2024). *ZED stereo depth cameras*. https://www.stereolabs.com/products/zed-2i

Suárez Martínez, J. (2026). *Vision-based human motion imitation for humanoid robots* [Undergraduate thesis, Universidad de los Andes]. Repositorio Uniandes. https://hdl.handle.net/1992/78007

Sun, S., Huang, H., & Li, C. (2025). Advancements in humanoid robot dynamics and learning-based locomotion control methods. *Intelligence & Robotics, 5*(3), 631–660. https://doi.org/10.20517/ir.2025.32

Todorov, E., Erez, T., & Tassa, Y. (2012). MuJoCo: A physics engine for model-based control. *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 5026–5033. https://doi.org/10.1109/IROS.2012.6386109

Unitree Robotics. (2024). *unitree_rl_gym* [GitHub repository]. https://github.com/unitreerobotics/unitree_rl_gym

Unitree Robotics. (n.d.). Unitree G1. Retrieved March 21, 2026, from https://www.unitree.com/g1

Wu, H., Xu, J., Wang, J., & Long, M. (2021). Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems 34* (pp. 22419–22430). Curran Associates. https://proceedings.neurips.cc/paper/2021/hash/bcc0d400288793e8bdcd7c19a8ac0c2b-Abstract.html

Wu, B., Zhang, T., Yang, C., Wen, J., Li, H., Ma, J., Chen, Z., & Wang, J. (2025). *SAGE: State-aware guided end-to-end policy for multi-stage sequential tasks via hidden Markov decision process* [Preprint]. arXiv. https://arxiv.org/abs/2509.19853

Zakka, K., Tassa, Y., & MuJoCo Menagerie Contributors. (2022). MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo. GitHub. https://github.com/google-deepmind/mujoco_menagerie

Zare, M., Kebria, P., Khosravi, A., & Nahavandi, S. (2024). A survey of imitation learning: Algorithms, recent developments, and challenges. *IEEE Transactions on Cybernetics, 54*(12), 7173–7186. https://ieeexplore.ieee.org/document/10602544

Zhao, T., Kumar, V., Levine, S., & Finn, C. (2023). Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*. https://arxiv.org/abs/2304.13705

Zhao, W., Queralta, J. P., & Westerlund, T. (2020). Sim-to-real transfer in deep reinforcement learning for robotics: A survey. *2020 IEEE Symposium Series on Computational Intelligence*, 737–744. https://ieeexplore.ieee.org/document/9308468

Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021). Informer: Beyond efficient transformer for long sequence time-series forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence, 35*(12), 11106–11115. https://doi.org/10.1609/aaai.v35i12.17325

Zhou, Y., Zhuang, X., Chen, Z., Zheng, Y., Lin, C., & Li, Z. (2025). MTIL: Encoding full history with Mamba for temporal imitation learning. *IEEE Robotics and Automation Letters*. https://doi.org/10.1109/LRA.2025.3615520